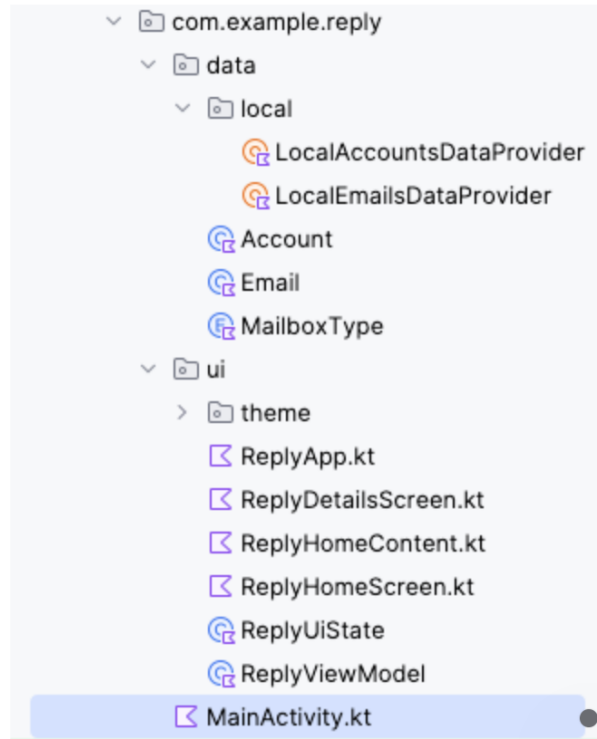


智能移动开发10

Important directories in Reply app



The data and UI layer of the Reply app project is separated into different directories. `ReplyViewModel`, `ReplyUiState`, and other composables are located in the `ui` directory. The `data` and `enum` classes that define the data layer and the data provider classes are located in the `data` directory.

The Reply app is initialized with data through the `initializeUIState()` method in the `ReplyViewModel`, which is executed in the `init` function.

ReplyViewModel.kt

```
...  
init {  
    initializeUIState()  
}  
  
private fun initializeUIState() {  
    var mailboxes: Map<MailboxType, List<Email>> =  
        LocalEmailsDataProvider.allEmails.groupBy { it.mailbox }  
    _uiState.value = ReplyUiState(  
        mailboxes = mailboxes,  
        currentSelectedEmail = mailboxes[MailboxType.Inbox]?.get(0)  
        ?: LocalEmailsDataProvider.defaultEmail  
    )  
}  
...
```

Build an adaptive app with dynamic navigation

English



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

```
viewModel.resetHomeScreenStates()  
},  
onEmailCardPressed = { email: Email ->  
    viewModel.updateDetailsScreenStates(  
        email = email  
    )  
},  
onDetailScreenBackPressed = {  
    viewModel.resetHomeScreenStates()  
},  
modifier = modifier  
}  
}
```

Other composables

- `ReplyHomeScreen.kt`: contains the screen composables for home screen, including navigation elements.
- `ReplyHomeContent.kt`: contains composables that define more detailed composables of the home screen.
- `ReplyDetailsScreen.kt`: contains screen composables and smaller composables for the details screen.

Feel free to go through each file in detail to gain more understanding of the composables before proceeding with the next section of the codelab.

Back

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

[Report a mistake](#)

Back

ReplyHomeScreen.kt

```
@Composable
fun ReplyHomeScreen(
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Int) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...
    if (replyUiState.isShowingHomepage) {
        ReplyAppContent(
            replyUiState = replyUiState,
            onTabPressed = onTabPressed,
            onEmailCardPressed = onEmailCardPressed,
            navigationItemContentList = navigationItemContentList,
            modifier = modifier
        )
    }
}
```

Next

You must now account for the scenario when the user is not at the home screen by showing the details screen.



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

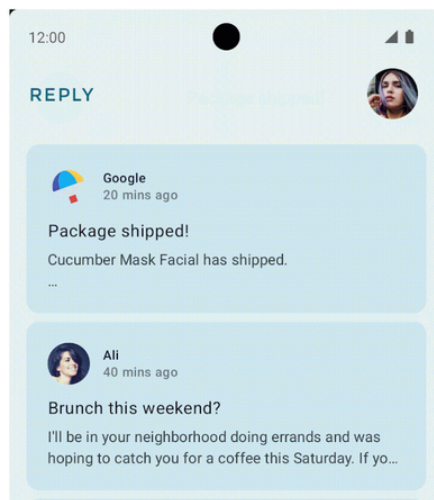
7 Implement adaptive navigation layout

8 Get the solution code

[Report a mistake](#)

Back

The `replyUiState` object is a state object. As such, when there is a change in the `isShowingHomepage` property of the `replyUiState` object, the `ReplyHomeScreen` composable is recomposed and the `if/else` statement is reevaluated at runtime. This approach supports navigation between different screens without the use of a `NavController` class.



Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

Use Window Size Classes

The `WindowSizeClass` API introduced for Compose makes the implementation of Material Design breakpoints simpler.

Window Size Classes introduces three categories of sizes: Compact, Medium, and Expanded, for both width and height.



Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

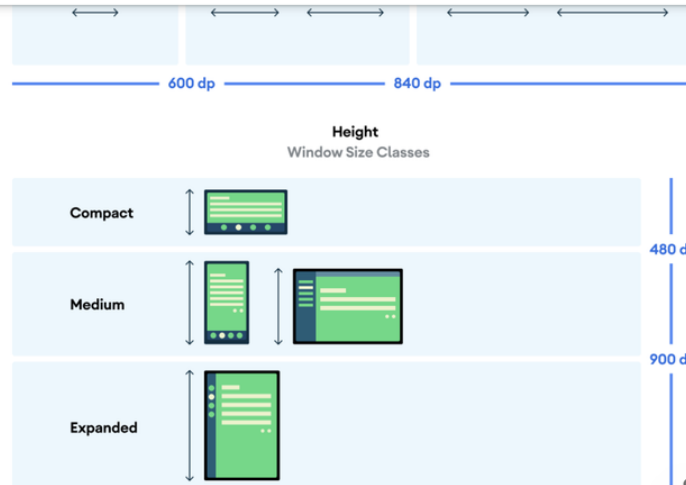
6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back



Complete the following steps to implement the `WindowSizeClass` API in the Reply app:

1. Add the `material3-window-size-class` dependency to the module `build.gradle.kts` file.

`build.gradle.kts`

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

You can use the `WindowWidthSizeClass` variable in the `MainActivity.kt` to determine which layout to display in various composables. Let's prepare the `ReplyApp` composable to receive this value.

6. In the `ReplyApp.kt` file, modify the `ReplyApp` composable to accept the `WindowWidthSizeClass` as the parameter and import the appropriate package.

ReplyApp.kt

```
...
import androidx.compose.material3.windowSizeClass.WindowWidthSizeClass
...

@Composable
fun ReplyApp(
    windowSize: WindowWidthSizeClass,
    modifier: Modifier = Modifier
) {
    ...
}
```

7. Pass the `windowSize` variable to the `ReplyApp` component in the `MainActivity.kt` file's `onCreate()` method.

MainActivity.kt

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

9. To change the app layouts based on the size of the screen, add a `when` statement in the `ReplyApp` composable based on the `WindowWidthSizeClass` value.

ReplyApp.kt

```
...

@Composable
fun ReplyApp(
    windowSize: WindowWidthSizeClass,
    modifier: Modifier = Modifier
) {
    val viewModel: ReplyViewModel = viewModel()
    val replyUiState = viewModel.uiState.collectAsState().value

    when (windowSize) {
        WindowWidthSizeClass.Compact -> {
        }
        WindowWidthSizeClass.Medium -> {
        }
        WindowWidthSizeClass.Expanded -> {
        }
        else -> {
        }
    }
}
```

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

Implement adaptive UI navigation

Currently, the [bottom navigation](#) is used for all screen sizes.



As previously discussed, this navigation element is not ideal because users can find it difficult to reach these essential navigation elements on larger screens. Fortunately, there are recommended patterns for different navigation elements for various window size classes in [navigation for responsive UIs](#). For the Reply app, you can implement the following elements:

Window size class	Few items
compact width	bottom navigation bar
medium width	navigation rail
expanded width	persistent navigation drawer (leading edge)

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

ReplyApp.kt

```
...
import com.example.reply.ui.utils.ReplyNavigationType
...
val navigationType: ReplyNavigationType
when (windowSize) {
    WindowWidthSizeClass.Compact -> {
        navigationType = ReplyNavigationType.BOTTOM_NAVIGATION
    }
    WindowWidthSizeClass.Medium -> {
        navigationType = ReplyNavigationType.NAVIGATION_RAIL
    }
    WindowWidthSizeClass.Expanded -> {
        navigationType = ReplyNavigationType.PERMANENT_NAVIGATION_DRAWER
    }
    else -> {
        navigationType = ReplyNavigationType.BOTTOM_NAVIGATION
    }
}
...
```

You can use the `navigationType` value in the `ReplyHomeScreen` composable. You can prepare for that by making it a parameter for the composable.

Next



1 Introduction

2 App overview

3 Starter code walkthrough

4 Change screens without a navigation graph

5 Run app on large screen devices

6 Make your layout adapt to different screen sizes

7 Implement adaptive navigation layout

8 Get the solution code

Report a mistake

Back

ReplyApp.kt

```
...
    ReplyHomeScreen(
        navigationType = navigationType,
        replyUiState = replyUiState,
        onTabPressed = { mailboxType: MailboxType ->
            viewModel.updateCurrentMailbox(mailboxType = mailboxType)
            viewModel.resetHomeScreenStates()
        },
        onEmailCardPressed = { email: Email ->
            viewModel.updateDetailsScreenStates(
                email = email
            )
        },
        onDetailScreenBackPressed = {
            viewModel.resetHomeScreenStates()
        },
        modifier = modifier
    )
...

```

Next, you can create a branch to show the app content with a navigation drawer when the user opens the app on an expanded screen and displays the homescreen.

Next



1 Introduction

2 Previews for different screen sizes

3 Implement adaptive content layout

4 Verify for different screen sizes

5 Add automated test for adaptive apps

6 Get the solution code

7 Conclusion

Report a mistake

Back

2. Add another preview for expanded screens by setting a large `widthDp` value in the `Preview` annotation parameter and specifying `WindowWidthSizeClass.Expanded` value as the parameter for the `ReplyApp` composable.

MainActivity.kt

```
...
@Preview(showBackground = true, widthDp = 1000)
@Composable
fun ReplyAppExpandedPreview() {
    ReplyTheme {
        Surface {
            ReplyApp(windowSize = WindowWidthSizeClass.Expanded)
        }
    }
}
...

```

3. Build the preview to view the following:

ReplyAppMediumPreview

Next



1 Introduction

2 Previews for different screen sizes

3 Implement adaptive content layout

4 Verify for different screen sizes

5 Add automated test for adaptive apps

6 Get the solution code

7 Conclusion

Report a mistake

Back

composable.

3. In `ReplyHomeScreen.kt`, add `contentType` as the parameter to the `ReplyHomeScreen` composable.

ReplyHomeScreen.kt

```
...
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReplyHomeScreen(
    navigationType: ReplyNavigationType,
    contentType: ReplyContentType,
    replyUiState: ReplyUiState,
    onTabPressed: (MailboxType) -> Unit,
    onEmailCardPressed: (Email) -> Unit,
    onDetailScreenBackPressed: () -> Unit,
    modifier: Modifier = Modifier
) {
    ...
}
```

4. Pass the `contentType` value to the `ReplyHomeScreen` composable.

ReplyApp.kt

...

Next



1 Introduction

2 Previews for different screen sizes

3 Implement adaptive content layout

4 Verify for different screen sizes

5 Add automated test for adaptive apps

6 Get the solution code

7 Conclusion

Report a mistake

Back

5. Add the `contentType` as a parameter for the `ReplyAppContent` composable.

ReplyHomeScreen.kt

```
...
@Composable
private fun ReplyAppContent(
    navigationType: ReplyNavigationType,
    contentType: ReplyContentType,
    replyUiState: ReplyUiState,
    onTabPressed: ((MailboxType) -> Unit),
    onEmailCardPressed: (Email) -> Unit,
    navigationItemContentList: List<NavigationItemContent>,
    modifier: Modifier = Modifier
) {
    ...
}
```

6. Pass the `contentType` value to the two `ReplyAppContent` composables.

ReplyHomeScreen.kt

```
...
    ReplyAppContent(
        navigationType = navigationType,
```

Next



1 Introduction

2 Previews for different screen sizes

3 Implement adaptive content layout

4 Verify for different screen sizes

5 Add automated test for adaptive apps

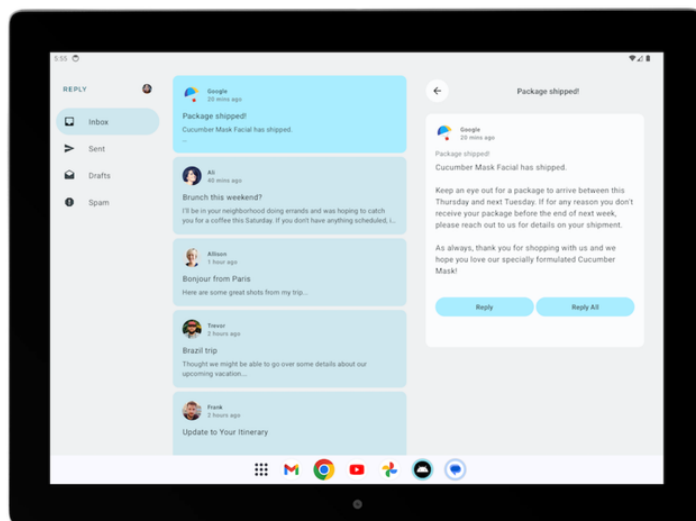
6 Get the solution code

7 Conclusion

Report a mistake

Back

9. Run your app on the tablet mode to see the screen below:



Next



1 Introduction

2 Previews for different screen sizes

3 Implement adaptive content layout

4 Verify for different screen sizes

5 Add automated test for adaptive apps

6 Get the solution code

7 Conclusion

Report a mistake

Back

13. Add an `isFullScreen` value as a parameter to the `ReplyEmailDetailsCard` composable.

ReplyDetailsScreen.kt

```
...  
@OptIn(ExperimentalMaterial3Api::class)  
@Composable  
private fun ReplyEmailDetailsCard(  
    email: Email,  
    mailboxType: MailboxType,  
    modifier: Modifier = Modifier,  
    isFullScreen: Boolean = false  
) {  
    ...  
}
```

14. Inside the `ReplyEmailDetailsCard` composable, only show the email subject text when the app is not in full screen, as the full screen layout already displays the email subject as the header. If it is full screen, add a spacer with height of `R.dimen.detail_content_padding_top`.

ReplyDetailsScreen.kt

Next



- 1 Introduction
- 2 Previews for different screen sizes
- 3 Implement adaptive content layout
- 4 Verify for different screen sizes
- 5 Add automated test for adaptive apps
- 6 Get the solution code
- 7 Conclusion

[Report a mistake](#)[Back](#)

5. In the `ReplyAppTest` class, create a test function to test for a compact size screen. Set the content of the `composeTestRule` with the `ReplyApp` composable and pass the `WindowSizeClass.Compact` as the `windowSize` argument.

Note: Adding a composable that accepts `WindowSizeClass` as an argument is a good practice to make testable code.

ReplyAppTest.kt

```
...  
@Test  
fun compactDevice_verifyUsingBottomNavigation() {  
    // Set up compact window  
    composeTestRule.setContent {  
        ReplyApp(  
            windowSize = WindowWidthSizeClass.Compact  
        )  
    }  
}
```



6. Assert that the bottom navigation element exists with the test tag. Call the extension function `onNodeWithTagForStringId` on the `composeTestRule` and pass the navigation element tag and call the `assertExists()` function.

[Next](#)

- 1 Introduction
- 2 Previews for different screen sizes
- 3 Implement adaptive content layout
- 4 Verify for different screen sizes
- 5 Add automated test for adaptive apps
- 6 Get the solution code
- 7 Conclusion

[Report a mistake](#)[Back](#)

```
...  
...  
}
```

4. Add a test to verify that a navigation rail element exists in the medium screens.

ReplyAppTest.kt

```
...  
@Test  
fun mediumDevice_verifyUsingNavigationRail() {  
    // Set up medium window  
    composeTestRule.setContent {  
        ReplyApp(  
            windowSize = WindowWidthSizeClass.Medium  
        )  
    }  
    // Navigation rail is displayed  
    composeTestRule.onNodeWithTagForStringId(  
        R.string.navigation_rail  
    ).assertExists()  
}
```



5. Add a test to verify that a navigation drawer element exists in the expanded screens.

[Next](#)



1 Before you begin

2 Get started

3 Plan to adapt Sports app for large screens

4 Create the expanded screen in code

5 Make app change to different layout based on window size

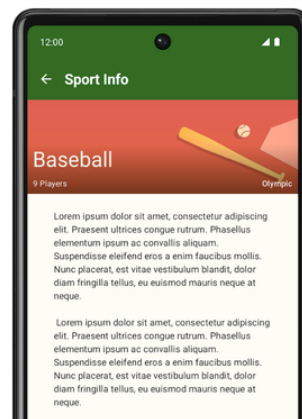
6 Get the solution code

Report a mistake

Back

To adapt the Sports app for large screens, you first want to establish the type of layout that best displays the app on large screens.

1. Start reviewing the current layouts available on the compact screen size. There are two screens — namely the list screen, which corresponds to the `SportsList` composable, and the detail screen, which corresponds to the `SportsDetail` composable.



Next



1 Before you begin

2 Get started

3 Plan to adapt Sports app for large screens

4 Create the expanded screen in code

5 Make app change to different layout based on window size

6 Get the solution code

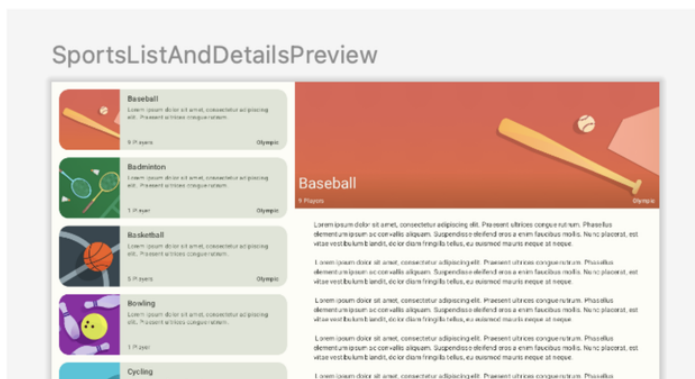
Report a mistake

Back

4. Create the expanded screen in code

Now that you have a clear view of how the expanded layout looks, let's translate that into code.

1. Create a composable for the expanded screen that shows both the list and the details screen. You can name it `SportsListAndDetails` and place it in the `SportsScreens.kt` file.
2. Create a composable preview to simplify verification of the UI for the `SportsListAndDetails` composable.



Next

Next step



1 Before you begin

2 Get started

3 Plan to adapt Sports app for large screens

4 Create the expanded screen in code

5 Make app change to different layout based on window size

6 Get the solution code

Report a mistake

Back

5. Make app change to different layout based on window size

To add the expanded screen composable to the app, complete the following tasks:

1. Add `androidx.compose.material3:material3-window-size-class` to the app's `build.gradle.kts`.
2. Calculate `windowSizeClass` in the `MainActivity` and pass the `widthSizeClass` to the `SportsApp` composable.
3. Create a new directory named `utils` and create a new file name `WindowStateUtils.kt`.
4. Add an `enum` class in `WindowStateUtils` to denote two different `contentType`. You can name them `ListOnly` and `ListAndDetail` types.
5. In the `SportsApp` composable, determine the `contentType`, based on the `widthSizeClass`.
6. Display the `SportsListAndDetails` composable when `contentType` is `ListAndDetail` and keep the previous behavior when the `contentType` is `ListOnly`.
7. For the `SportsAppBar` composable, change the behavior so that the back button doesn't appear and the app bar shows **Sports** when the screen expands in the list page.
8. Verify the UI and navigation capabilities for both compact and expanded screens using the resizable emulator.

Next

Next step



1 Before you begin

2 Overview

3 Gather the content for the app

4 Sketch the app screens

5 Plan the steps

6 Implement the app

7 Get feedback from a user (Optional)

8 Test the app with Resizable Emulator

Report a mistake

- How to build an app with multiple screens
- How to build an adaptive layout so that your app works well across a range of screen sizes
- The importance of app architecture
- How to ensure your app adheres to the activity lifecycle without wasting unnecessary resources
- How to preserve user state across device configuration changes

That was a lot to learn!

Following the codelab steps helped you learn these new concepts to a certain degree, but you will gain a deeper level of understanding by applying your knowledge and skills to a new app.

You will use your knowledge when you run into new situations, bugs, or times when you need to design a solution yourself. You can always refer back to previous apps and codelabs to quickly remind yourself what you learned and put that into action.

You will learn more if you try to tackle these problems on your own, which will more closely resemble what the "real world" is like as a developer. It's a skill in itself to know how to break down a problem, troubleshoot it, and search for resources to find a solution. For that reason, you're encouraged to work on these projects at the end of each unit. With this motivation and context in mind, check out the following prompt for the Unit 4 project.

Prerequisites

- Ability to create and run a project in Android Studio
- Familiar with the basics of the Kotlin programming language

Next